

0618 上午 Vue3 课件学习梳理

1. 这节课到底在学什么

这节课围绕一个核心目标：从传统 HTML + CSS + JS 写法，过渡到现代 Vue 3 工程化开发。

你需要掌握三条主线：

1. 工程怎么跑起来：Node.js、npm、Vite、项目目录结构。
2. 页面怎么由数据驱动：ref、reactive、v-if、v-for、computed。
3. 组件之间怎么配合：父组件通过 Props 传数据，子组件通过 Emits 回传事件。

课后项目主要考察：

1. 能不能用 v-if 和 v-for 做一个技能标签管理器。
2. 能不能用 Props 和 Emits 做一个打卡学时累加系统。

2. Node.js、npm、Vite 的关系

传统网页开发里，浏览器直接打开 index.html 就能运行。但 Vue 项目不是这样，因为浏览器不能直接理解 .vue 单文件组件，也不能自动处理模块导入、依赖安装和打包。

所以需要这几个工具：

工具	作用
Node.js	让 JavaScript 可以在本地电脑运行，为前端工程提供运行环境
npm	包管理器，用来安装项目依赖
Vite	Vue 项目的开发服务器和打包工具

常用命令：

```
# 查看 Node 是否安装成功
node -v

# 查看 npm 是否可用
npm -v

# 配置国内 npm 镜像
npm config set registry https://registry.npmmirror.com

# 创建 Vue3 + Vite 项目
npm create vite@latest vue3-demo-project -- --template vue

# 安装依赖
npm install

# 启动开发服务器
```

```
npm run dev
```

如果 PowerShell 提示 `npm.ps1` 禁止运行，可以用：

```
npm.cmd run dev
```

3. Vue3 项目目录怎么看

当前项目的主要结构：

```
vue3-demo-project/
├── node_modules/           # npm 安装的依赖，不提交到 Git
├── public/                 # 公共静态资源，打包时原样复制
├── src/                    # 核心开发目录
│   ├── assets/            # 图片、样式等资源
│   ├── components/        # 可复用组件
│   ├── App.vue            # 根组件，页面主体从这里开始
│   ├── main.js            # 项目入口 JS
│   └── style.css          # 全局样式
├── index.html              # SPA 唯一 HTML 入口
├── package.json            # 依赖和脚本配置
└── vite.config.js         # Vite 配置
```

最重要的加载流程：

```
浏览器打开 index.html
      ↓
index.html 里有 <div id="app"></div>
      ↓
加载 src/main.js
      ↓
main.js 引入 App.vue
      ↓
createApp(App).mount('#app')
      ↓
Vue 把 App.vue 渲染到 #app 容器里
```

一句话记忆：

`index.html` 是空壳，`main.js` 负责挂载，`App.vue` 才是真正显示出来的页面内容。

4. Composition API：ref 和 reactive

Vue3 常用 `<script setup>` 写组合式 API。

ref

ref 常用于数字、字符串、布尔值，也可以包对象和数组。

重点：

1. 在 `<script setup>` 里读写时要用 `.value`。

2. 在 `<template>` 里会自动解包，不写 `.value`。

```
<script setup>
import { ref } from 'vue'

const count = ref(0)

function add() {
  count.value++
}
</script>

<template>
  <p>{{ count }}</p>
  <button @click="add">加 1</button>
</template>
```

reactive

`reactive` 常用于对象、数组、`Map`、`Set` 这类复杂数据。

重点：

- 1. 不需要 `.value`。
- 2. 不建议直接解构，否则容易丢失响应式。

```
<script setup>
import { reactive } from 'vue'

const student = reactive({
  name: '张三',
  age: 18,
})

function changeName() {
  student.name = '李四'
}
</script>
```

ref 和 reactive 对比

对比项	ref	reactive
常用场景	基本类型、简单状态	对象、数组、复杂状态
JS 中访问	<code>count.value</code>	<code>student.name</code>
模板中访问	<code>{{ count }}</code>	<code>{{ student.name }}</code>
底层理解	把值包成响应式对象	用 Proxy 代理整个对象

5. v-if：条件渲染

`v-if` 根据条件决定元素是否进入 DOM。

它不是简单隐藏，而是条件为假时把元素从 DOM 中销毁；条件为真时再重新创建。

```
<script setup>
import { ref } from 'vue'

const isLoggedIn = ref(false)
</script>

<template>
  <p v-if="isLoggedIn">欢迎回来</p>
  <p v-else>请先登录</p>

  <button @click="isLoggedIn = !isLoggedIn">
    切换状态
  </button>
</template>
```

注意：

`v-else` 必须紧跟在 `v-if` 后面，中间不能插入其他元素。

6. v-for：列表循环

`v-for` 用来根据数组批量渲染页面。

```
<script setup>
import { ref } from 'vue'

const skills = ref(['Vue3', 'SpringBoot', 'Git'])
</script>

<template>
  <ul>
    <li v-for="(item, index) in skills" :key="item">
      {{ index + 1 }}. {{ item }}
    </li>
  </ul>
</template>
```

重点：

1. `item` 是当前项。
2. `index` 是下标，从 0 开始。
3. `:key` 必须尽量唯一，能用业务唯一值时优先用业务唯一值。

7. computed：计算属性

`computed` 用于根据已有响应式数据推导新数据。

它有缓存：依赖不变时，不会重复计算。

```

<script setup>
import { ref, computed } from 'vue'

const qty = ref(1)

const total = computed(() => {
  return qty.value * 100
})
</script>

<template>
  <p>数量：{{ qty }}</p>
  <p>总价：{{ total }}</p>
  <button @click="qty++">加 1 件</button>
</template>

```

适合用 `computed` 的情况：

1. 总价计算。
2. 分数合计。
3. 列表过滤。
4. 时间、金额、状态文案格式化。

8. Props 和 Emits：组件通信

Vue 组件通信遵循单向数据流：

```

父组件 --Props--> 子组件
父组件 <--Emits-- 子组件

```

子组件接收 Props

```

<script setup>
defineProps({
  taskName: String,
})
</script>

<template>
  <p>{{ taskName }}</p>
</template>

```

子组件发射 Emits

```

<script setup>
const emit = defineEmits(['taskCompleted'])

function completeTask() {
  emit('taskCompleted', 30)
}
</script>

```

```
<template>
  <button @click="completeTask">完成打卡</button>
</template>
```

父组件监听子组件事件

```
<script setup>
import { ref } from 'vue'
import TaskItem from './components/TaskItem.vue'

const totalTime = ref(0)

function handleTaskCompleted(minutes) {
  totalTime.value += minutes
}
</script>

<template>
  <p>累计学时：{{ totalTime }} 分钟</p>
  <TaskItem
    task-name="Vue 3 组件通信训练"
    @task-completed="handleTaskCompleted"
  />
</template>
```

9. 课后练习 1：伴学技能标签管理器

要求拆解：

1. 用 `ref` 声明布尔变量 `showWall`。
2. 用 `ref` 声明数组 `mySkills`，默认包含 `Vue3`、`SpringBoot`、`Git`。
3. 用 `v-if` 判断技能墙是否显示。
4. 显示时用 `v-for` 渲染技能列表。
5. 隐藏时显示“技能墙已被隐藏”。
6. 核心逻辑要写中文注释。

当前项目已实现位置：

```
src/App.vue
```

关键代码：

```
<script setup>
import { ref } from 'vue'

// 练习 1：控制技能墙是否显示
const showWall = ref(true)

// 练习 1：技能数组用于 v-for 循环渲染
const mySkills = ref(['Vue3', 'SpringBoot', 'Git'])
</script>
```

10. 课后练习 2：打卡学时互传系统

要求拆解：

1. 创建 `src/components/TaskItem.vue`。
2. 子组件通过 `Props` 接收 `taskName`。
3. 子组件按钮点击后触发 `taskCompleted`。
4. 子组件向父组件传递数值 `30`。
5. 父组件声明 `totalTime`，默认 `0`。
6. 父组件监听 `taskCompleted`，收到分钟数后累加。
7. 页面实时显示累计学时。
8. 两个组件源码必须能直接复制运行。

当前项目已实现位置：

```
src/App.vue  
src/components/TaskItem.vue
```

核心通信逻辑：

```
App.vue 把 task-name 传给 TaskItem.vue  
TaskItem.vue 点击按钮 emit('taskCompleted', 30)  
App.vue 监听 @task-completed  
App.vue 执行 handleTaskCompleted(minutes)  
totalTime 累加并更新页面
```

11. 当前项目运行与验证

进入项目目录：

```
cd D:\code\2026.06-07\vue3-demo-project\vue3-demo-project
```

启动开发服务器：

```
npm.cmd run dev
```

构建检查：

```
npm.cmd run build
```

浏览器验证点：

1. 点击“隐藏技能墙”，技能列表消失，显示“技能墙已被隐藏”。
2. 再点击“显示技能墙”，技能列表恢复。
3. 点击“完成打卡”，累计学时每次增加 `30` 分钟。
4. 子组件中能看到父组件传入的任务名。

12. 学习时最容易混的点

容易混的点	正确理解
ref 在 JS 中为什么要 .value	因为 ref 把值包进了对象
模板里为什么不用 .value	Vue 模板会自动解包
v-if 是隐藏吗	不是，只要条件为假就会销毁 DOM
v-for 为什么要写 :key	帮助 Vue 精准复用 DOM，提高渲染效率
子组件能不能直接改父组件数据	不应该直接改，要通过 emit 通知父组件
computed 和函数有什么区别	computed 有依赖追踪和缓存

13. 复习顺序建议

1. 先看懂项目启动流程：index.html、main.js、App.vue。
2. 再练 ref：能做到点击按钮改变页面数字。
3. 再练 v-if：能做到显示和隐藏内容。
4. 再练 v-for：能把数组渲染成列表。
5. 再看 computed：理解“由已有数据推导新数据”。
6. 最后练组件通信：父传子用 Props，子传父用 Emits。

只要能独立讲清楚“数据变了，页面为什么会跟着变”，这节课的 Vue3 核心就基本吃透了。